```
import pandas as pd
import matplotlib.pyplot as plt

# Sample dataset
data = {
    'Year': [2016, 2017, 2018, 2019, 2020,
             2021, 2022, 2023, 2024, 2025],
    'Job_Postings': [1200, 1500, 1900, 2500, 3000,
                     4200, 5500, 6000, 7200, 8500]
}

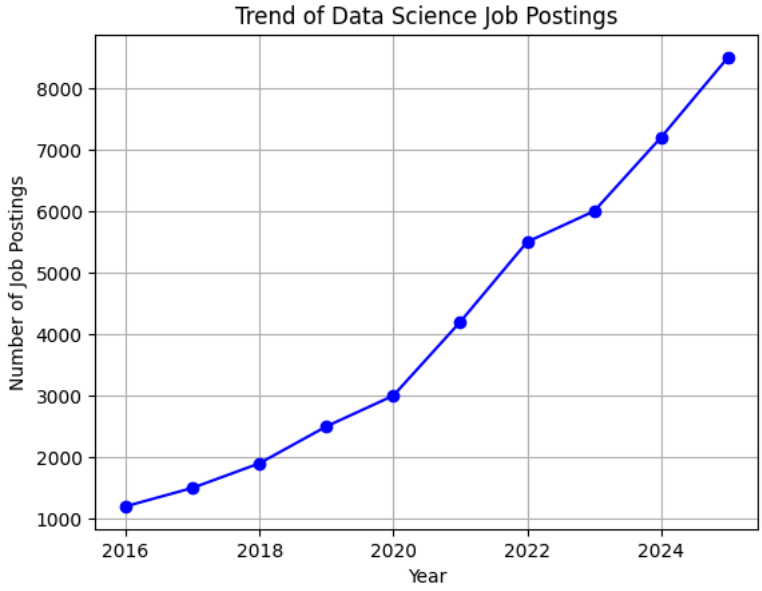
df = pd.DataFrame(data)

# Display the dataset
print("Data Science Job Postings:")
print(df)

# Visualize the trend
plt.plot(df['Year'], df['Job_Postings'],
         marker='o', color='blue')

plt.title("Trend of Data Science Job Postings")
plt.xlabel("Year")
plt.ylabel("Number of Job Postings")
plt.grid(True)
plt.show()
```

```
Data Science Job Postings:
   Year  Job_Postings
0  2016          1200
1  2017          1500
2  2018          1900
3  2019          2500
4  2020          3000
5  2021          4200
6  2022          5500
7  2023          6000
8  2024          7200
9  2025          8500
```



Start coding or generate with AI.

```
import pandas as pd
import matplotlib.pyplot as plt

# Sample dataset
data = {
    'Job_Role': [
        'Data Analyst',
        'Data Engineer',
        'Data Scientist',
        'ML Engineer',
        'Business Analyst',
        'Data Analyst',
```

```
        'Data Scientist',
        'Data Engineer',
        'Data Analyst',
        'ML Engineer'
    ]
}

df = pd.DataFrame(data)

# Count each job role
role_count = df['Job_Role'].value_counts()

print("Distribution of Data Science Roles:")
print(role_count)

# Bar chart
role_count.plot(kind='bar')

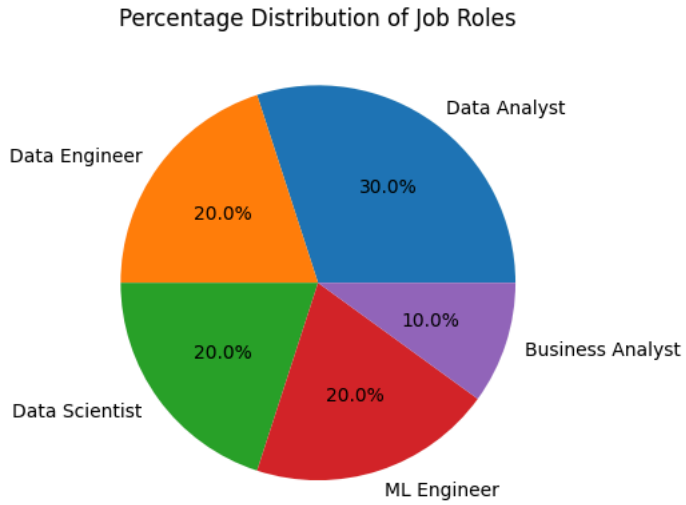
plt.title("Distribution of Data Science Job Roles")
plt.xlabel("Job Role")
plt.ylabel("Number of People")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Pie chart
role_count.plot(kind='pie', autopct='%1.1f%%')

plt.title("Percentage Distribution of Job Roles")
plt.ylabel("")
plt.show()
```

```
Distribution of Data Science Roles:
Job_Role
Data Analyst        3
Data Engineer       2
Data Scientist      2
ML Engineer         2
Business Analyst    1
Name: count, dtype: int64
```






```
import pandas as pd
import json

# 1. Structured data
students = {
    'ID': [1, 2, 3],
    'Name': ['Arun', 'Priya', 'Kumar'],
    'Mark': [85, 90, 78]
}

df = pd.DataFrame(students)

print("STRUCTURED DATA")
print(df)

# 2. Semi-structured data
data = {
    "student1": {"name": "Arun", "mark": 85},
    "student2": {"name": "Priya", "mark": 90}
}

print("\nSEMI-STRUCTURED DATA")
print(json.dumps(data, indent=4))

# 3. Unstructured data
```

```
documents = [
    "Data science uses Python.",
    "Machine learning analyzes data.",
    "Images and audio are examples of unstructured data."
]

print("\nUNSTRUCTURED DATA")
for text in documents:
    print(text)
```

```
STRUCTURED DATA
   ID   Name  Mark
0   1   Arun    85
1   2  Priya    90
2   3  Kumar    78

SEMI-STRUCTURED DATA
{
    "student1": {
        "name": "Arun",
        "mark": 85
    },
    "student2": {
        "name": "Priya",
        "mark": 90
    }
}

UNSTRUCTURED DATA
Data science uses Python.
Machine learning analyzes data.
Images and audio are examples of unstructured data.
```

```
from cryptography.fernet import Fernet

# Generate a secret key
key = Fernet.generate_key()

# Create Fernet object
cipher = Fernet(key)

# Original sensitive data
data = "Name: Arun, Account Number: 123456"

print("Original Data:", data)

# Encrypt the data
encrypted_data = cipher.encrypt(data.encode())
print("Encrypted Data:", encrypted_data)

# Decrypt the data
decrypted_data = cipher.decrypt(encrypted_data).decode()
print("Decrypted Data:", decrypted_data)
```

```
Original Data: Name: Arun, Account Number: 123456
Encrypted Data: b'gAAAAABqzdassHSgLAf4SiQcMaDT3QaaJnuaK8_Sm54hf2yLYG2GzhdJWM7aDhDIQJ_wDdpze2JwkfLxbXPeIjaTYrMEx96t8bepcO-eIh
Decrypted Data: Name: Arun, Account Number: 123456
```

```
# Generate key and encrypt data
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
token = f.encrypt(b"Rajalakshmi Engineering College")
token
b'...'
f.decrypt(token)
b'Rajalakshmi Engineering College'
key = Fernet.generate_key()
cipher_suite = Fernet(key)
plain_text = b"Rajalakshmi Engineering College."
cipher_text = cipher_suite.encrypt(plain_text)
# Decrypt data
decrypted_text = cipher_suite.decrypt(cipher_text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted_text)
```

```
Original Data: b'Rajalakshmi Engineering College.'
Encrypted Data: b'gAAAAABqzdtrsK3KG4mOAQhaNGUm2_058oITabsUmtd4rvx4pTfUZJRFYuZSzjgaB41CifmLywsgBoRJX4lko42YtgUwje8wp2EYq2TceE
```

```
Decrypted Data: b'Rajalakshmi Engineering College.'
```

```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files

# Upload CSV file
uploaded = files.upload()

# Read the dataset
filename = next(iter(uploaded))
df = pd.read_csv(filename)

# Display the dataset
print("First 5 Rows:")
print(df.head())

print("\nDataset Information:")
print(df.info())

print("\nDataset Shape:")
print(df.shape)

# Data preprocessing
print("\nMissing Values:")
print(df.isnull().sum())

# Remove duplicate rows
df = df.drop_duplicates()

# Convert Date column into date format
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True, errors='coerce')

# Convert numeric columns
df['Sales'] = pd.to_numeric(df['Sales'], errors='coerce')
df['Quantity'] = pd.to_numeric(df['Quantity'], errors='coerce')

# Fill missing numeric values with column averages
df['Sales'] = df['Sales'].fillna(df['Sales'].mean())
df['Quantity'] = df['Quantity'].fillna(df['Quantity'].mean())

# Remove rows with missing important values
df = df.dropna(subset=['Date', 'Product', 'Region'])

print("\nCleaned Dataset:")
print(df.head())

print("\nDataset After Preprocessing:")
print(df.info())

# Data analysis
print("\nTotal Sales:", df['Sales'].sum())
print("Total Quantity Sold:", df['Quantity'].sum())

print("\nSales by Product:")
print(df.groupby('Product')['Sales'].sum())

print("\nSales by Region:")
print(df.groupby('Region')['Sales'].sum())

# Visualization 1: Sales by Product
df.groupby('Product')['Sales'].sum().plot(
    kind='bar', title='Total Sales by Product'
)
plt.xlabel('Product')
plt.ylabel('Sales')
plt.tight_layout()
plt.show()

# Visualization 2: Sales by Region
df.groupby('Region')['Sales'].sum().plot(
    kind='pie', autopct='%1.1f%%',
    title='Sales Distribution by Region'
)
plt.ylabel('')
plt.show()

# Visualization 3: Sales over Time
df.groupby('Date')['Sales'].sum().sort_index().plot(
```

```
    kind='line', marker='o', title='Sales Over Time'
)
plt.xlabel('Date')
plt.ylabel('Sales')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Visualization 4: Quantity Sold by Product
df.groupby('Product')['Quantity'].sum().plot(
    kind='bar', title='Quantity Sold by Product'
)
plt.xlabel('Product')
plt.ylabel('Quantity')
plt.tight_layout()
plt.show()

# Save cleaned dataset
df.to_csv('cleaned-sales-data.csv', index=False)

print("\nCleaned dataset saved successfully!")

# Download cleaned dataset
files.download('cleaned-sales-data.csv')
```

```
Choose files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving sales-data (1).csv to sales-data (1) (1).csv
First 5 Rows:
         Date    Product  Sales  Quantity Region
0  01-01-2023  Product A    200         4  North
1  02-01-2023  Product B    150         3  South
2  03-01-2023  Product A    220         5  North
3  04-01-2023  Product C    300         6   East
4  05-01-2023  Product B    180         4   West

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16 entries, 0 to 15
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Date      16 non-null     object
 1   Product   16 non-null     object
 2   Sales     16 non-null     int64
 3   Quantity  16 non-null     int64
 4   Region    16 non-null     object
dtypes: int64(2), object(3)
memory usage: 772.0+ bytes
None

Dataset Shape:
(16, 5)

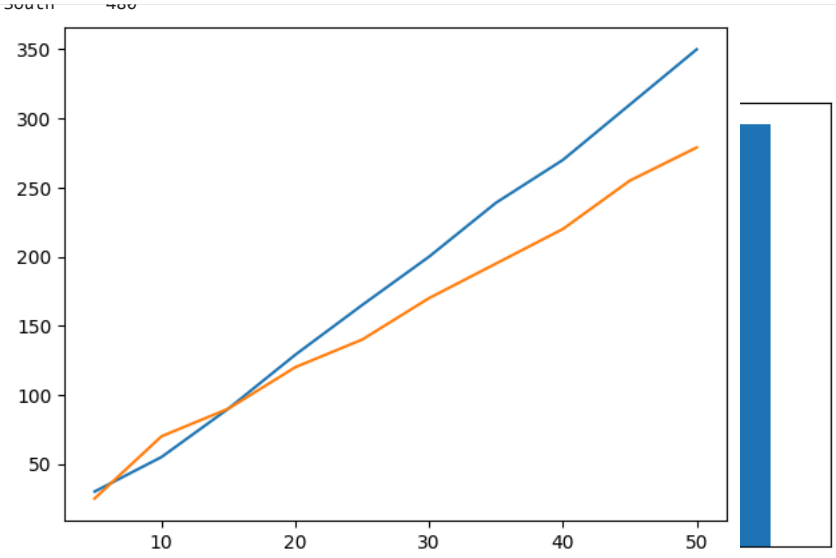
Missing Values:
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64

Cleaned Dataset:
        Date    Product  Sales  Quantity Region
0 2023-01-01  Product A    200         4  North
1 2023-01-02  Product B    150         3  South
2 2023-01-03  Product A    220         5  North
3 2023-01-04  Product C    300         6   East
4 2023-01-05  Product B    180         4   West

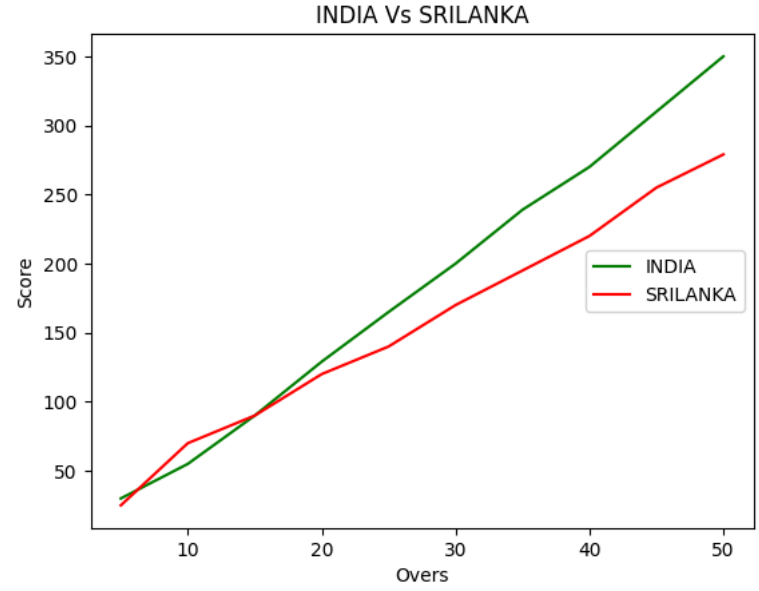
Dataset After Preprocessing:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16 entries, 0 to 15
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Date      16 non-null     datetime64[ns]
 1   Product   16 non-null     object
 2   Sales     16 non-null     int64
 3   Quantity  16 non-null     int64
 4   Region    16 non-null     object
dtypes: datetime64[ns](1), int64(2), object(2)
```

```
import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
```

```
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
cricket.plot(Overs,Indian_Score)
cricket.plot(Overs,Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
```

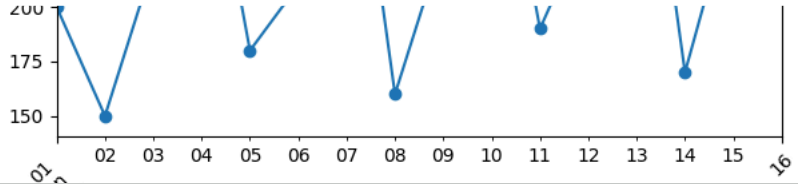


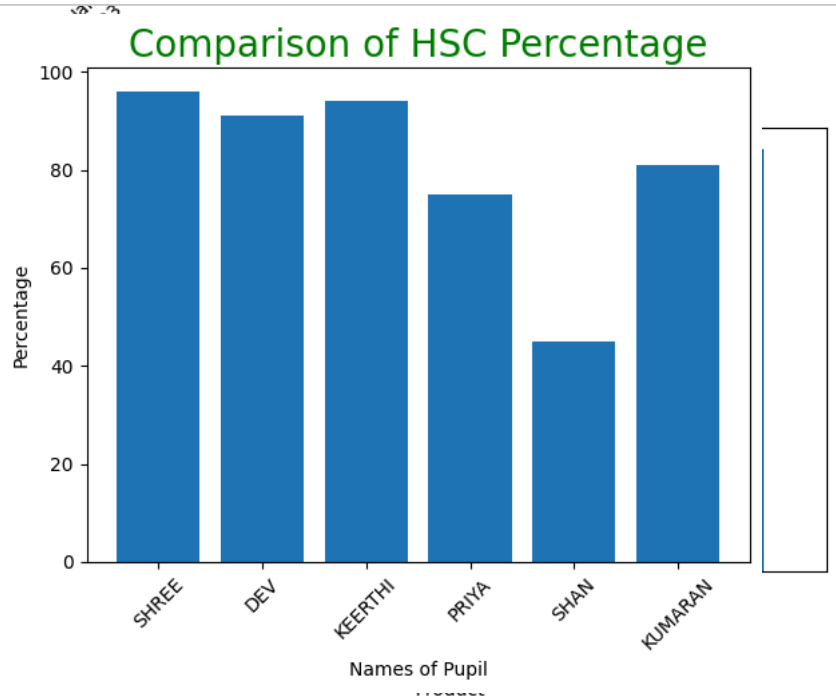

```
/tmp/ipykernel_5779/506286342.py:11: UserWarning: No artists with labels found to put in legend.  Note that artists whose la
  cricket.legend()
<matplotlib.legend.Legend at 0x7b360693afd0>
```



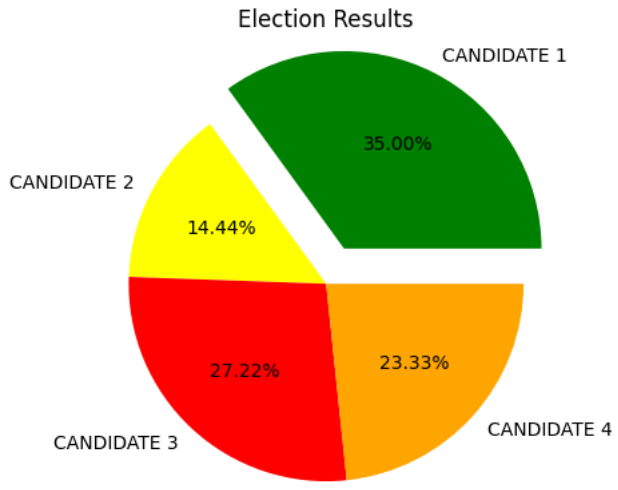



```
import matplotlib.pyplot as hscmark
import numpy as np
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```






```
import matplotlib.pyplot as election
# Election data
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)
# Plotting the pie chart
election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct='%0.2f%%')
election.title('Election Results')
election.show()
```




```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg

# Download the required data
nltk.download('gutenberg')
nltk.download('punkt')
nltk.download('punkt_tab')
sample = gutenberg.raw("austen-emma.txt")
token = word_tokenize(sample)
wlist = []
for i in range(50):
    wlist.append(token[i])
wordfreq = [wlist.count(w) for w in wlist]
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

```
[nltk_data] Downloading package gutenberg to /root/nltk_data...
[nltk_data]   Unzipping corpora/gutenberg.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816', 1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1
```

```
import pandas as pd
import numpy as np

df = pd.read_csv('/content/gemini-code-1791879161455.txt')

print(df.head())
print(df.info())
print(f"Shape: {df.shape}")
print(df.describe())
print(df.isnull().sum())
print(df['Outcome'].value_counts())
print(df['Outcome'].value_counts(normalize=True) * 100)

zero_counts = (df[['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']] == 0).sum()
print(zero_counts)
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               15 non-null     int64
 1   Glucose                   15 non-null     int64
 2   BloodPressure             15 non-null     int64
 3   SkinThickness             15 non-null     int64
 4   Insulin                   15 non-null     int64
 5   BMI                       15 non-null     float64
 6   DiabetesPedigreeFunction  15 non-null     float64
 7   Age                       15 non-null     int64
 8   Outcome                   15 non-null     int64
dtypes: float64(2), int64(7)
memory usage: 1.2 KB
None
Shape: (15, 9)
       Pregnancies     Glucose  BloodPressure  SkinThickness    Insulin  \
count    15.000000   15.000000      15.000000      15.000000   15.00000
mean      4.933333  136.333333      65.066667      16.066667  127.60000
std       3.594970   38.384645      22.839398      16.662476  245.20656
min       0.000000   78.000000       0.000000       0.000000    0.00000
25%       1.500000  112.500000      62.000000       0.000000    0.00000
50%       5.000000  137.000000      70.000000      19.000000    0.00000
75%       8.000000  167.000000      74.000000      30.500000  131.00000
max      10.000000  197.000000      96.000000      45.000000  846.00000

             BMI  DiabetesPedigreeFunction        Age    Outcome
count  15.000000                 15.000000  15.000000  15.000000
mean   29.046667                  0.548800  39.333333   0.600000
std     9.737693                  0.586514  12.921005   0.507093
min     0.000000                  0.134000  21.000000   0.000000
25%    26.200000                  0.196000  30.000000   0.000000
50%    30.100000                  0.351000  33.000000   1.000000
75%    34.450000                  0.607000  52.000000   1.000000
max    43.100000                  2.288000  59.000000   1.000000
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
```

```
import pandas as pd
import matplotlib.pyplot as plt
```

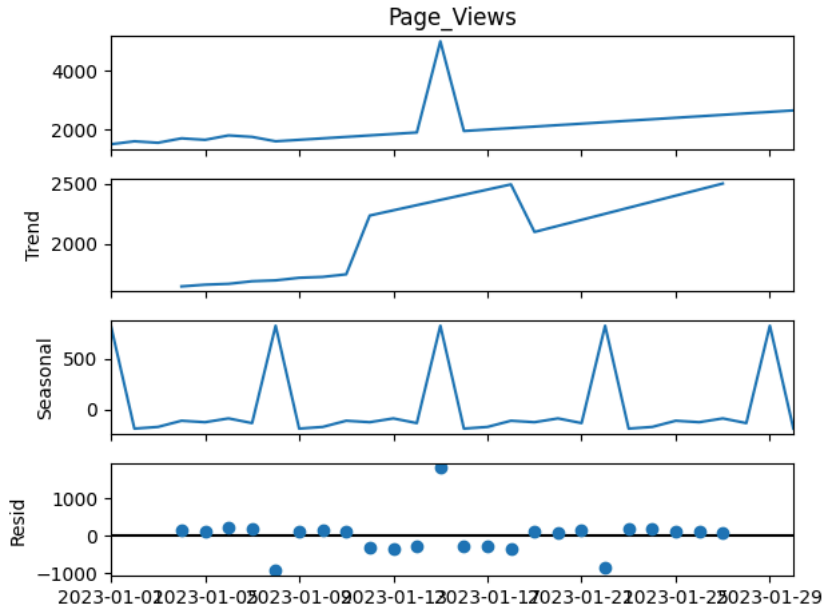
```
from statsmodels.tsa.seasonal import seasonal_decompose

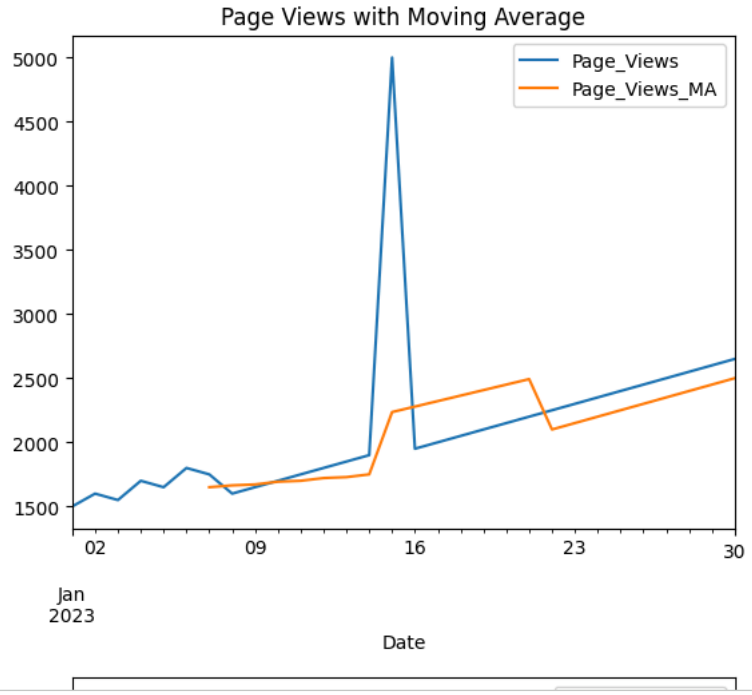
data = pd.read_csv('/content/gemini-code-1791879782319.txt', parse_dates=['Date'], index_col='Date')

decomposition = seasonal_decompose(data['Page_Views'], model='additive', period=7)
decomposition.plot()
plt.show()

data['Page_Views_MA'] = data['Page_Views'].rolling(window=7).mean()
data[['Page_Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()

data['Page_Views_Z'] = (data['Page_Views'] - data['Page_Views'].mean()) / data['Page_Views'].std()
anomalies = data[data['Page_Views_Z'].abs() > 2]
plt.plot(data.index, data['Page_Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page_Views'], color='red', label='Anomalies')
plt.legend()
plt.show()
```






```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)
population = np.random.normal(loc=170, scale=10, size=10000)

print(f"Population Mean: {np.mean(population):.2f}")
print(f"Population Std Dev: {np.std(population):.2f}")
```

```
sample_size = 30
sample_mean = np.mean(np.random.choice(population, size=sample_size))
print(f"Single Sample Mean (n={sample_size}): {sample_mean:.2f}")

num_samples = 1000
sample_means = [np.mean(np.random.choice(population, size=sample_size)) for _ in range(num_samples)]

print(f"Sampling Distribution Mean: {np.mean(sample_means):.2f}")
print(f"Sampling Distribution Std Error: {np.std(sample_means):.2f}")
print(f"Theoretical Standard Error: {np.std(population) / np.sqrt(sample_size):.2f}")

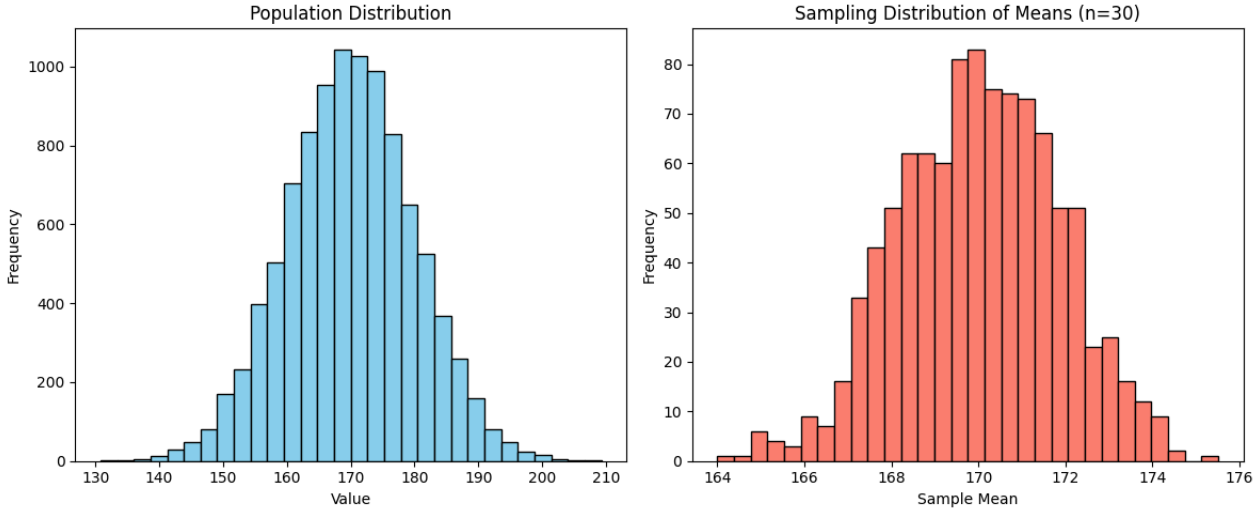
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.hist(population, bins=30, color='skyblue', edgecolor='black')
plt.title("Population Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")

plt.subplot(1, 2, 2)
plt.hist(sample_means, bins=30, color='salmon', edgecolor='black')
plt.title(f"Sampling Distribution of Means (n={sample_size})")
plt.xlabel("Sample Mean")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

```
Population Mean: 169.98
Population Std Dev: 10.03
Single Sample Mean (n=30): 170.46
Sampling Distribution Mean: 170.02
Sampling Distribution Std Error: 1.84
Theoretical Standard Error: 1.83
```




```
import numpy as np
from statsmodels.stats.weightstats import ztest

data = np.array([148, 153, 146, 155, 152, 149, 151, 147, 154, 150,
                 152, 148, 153, 149, 151, 156, 145, 150, 152, 148])

hypothesized_mean = 150

z_stat, p_value = ztest(data, value=hypothesized_mean)

alpha = 0.05

print(f"Sample Mean: {np.mean(data):.2f}")
print(f"Z-statistic: {z_stat:.4f}")
print(f"P-value: {p_value:.4f}")

if p_value < alpha:
    print("Reject Null Hypothesis: Average weight differs significantly from 150 grams.")
else:
    print("Fail to Reject Null Hypothesis: No significant difference from 150 grams.")
```

```
Sample Mean: 150.45
Z-statistic: 0.6749
```

```
P-value: 0.4998
Fail to Reject Null Hypothesis: No significant difference from 150 grams.
```

```
import matplotlib.pyplot as plt
import seaborn as sns

iris = sns.load_dataset('iris')

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

axes[0, 0].hist(iris['sepal_length'], bins=15, color='skyblue', edgecolor='black')
axes[0, 0].set_title('Distribution of Sepal Length')
axes[0, 0].set_xlabel('Sepal Length (cm)')
axes[0, 0].set_ylabel('Frequency')

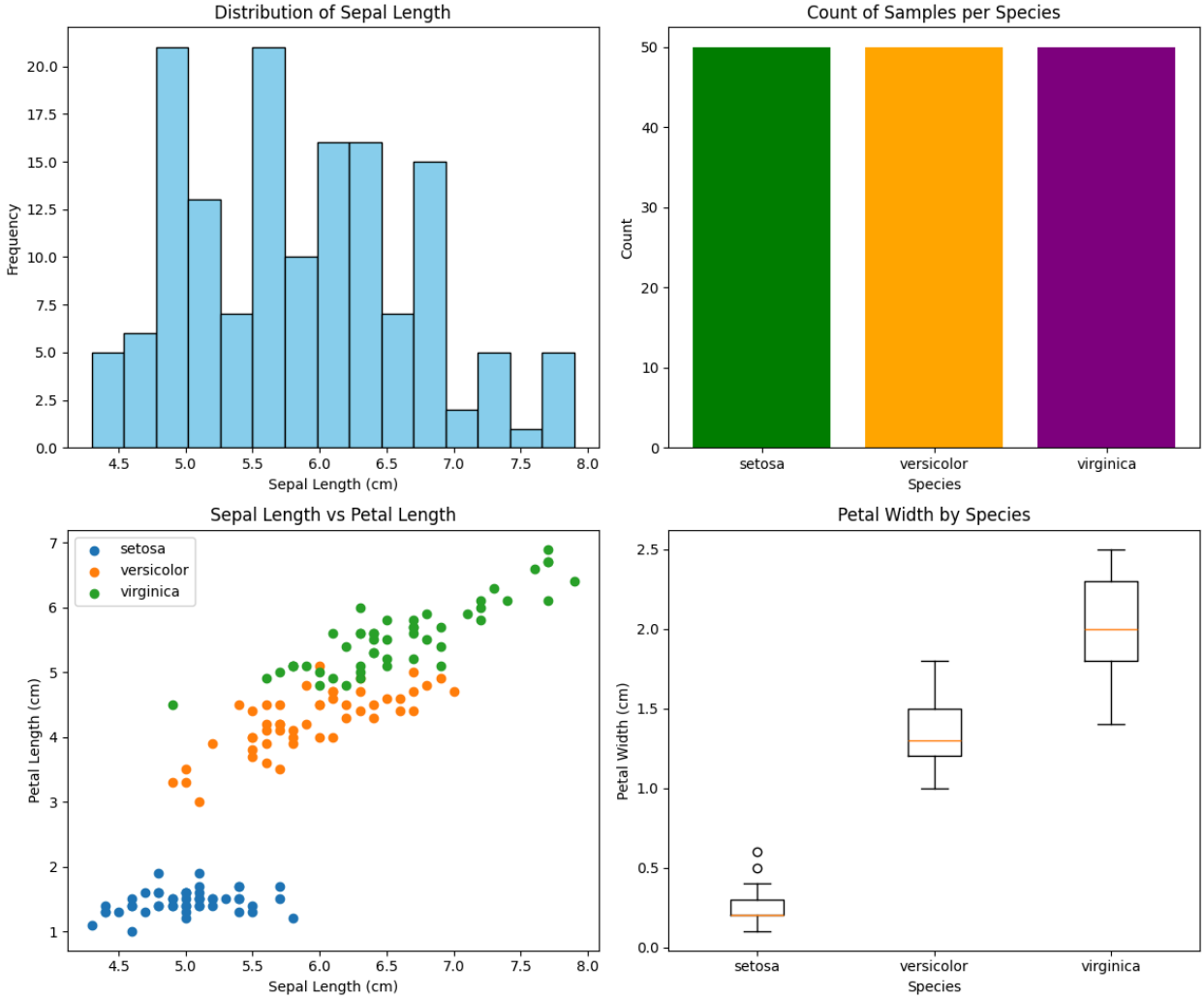
species_counts = iris['species'].value_counts()
axes[0, 1].bar(species_counts.index, species_counts.values, color=['green', 'orange', 'purple'])
axes[0, 1].set_title('Count of Samples per Species')
axes[0, 1].set_xlabel('Species')
axes[0, 1].set_ylabel('Count')

for species, group in iris.groupby('species'):
    axes[1, 0].scatter(group['sepal_length'], group['petal_length'], label=species)
axes[1, 0].set_title('Sepal Length vs Petal Length')
axes[1, 0].set_xlabel('Sepal Length (cm)')
axes[1, 0].set_ylabel('Petal Length (cm)')
axes[1, 0].legend()

species_data = [group['petal_width'].values for _, group in iris.groupby('species')]
axes[1, 1].boxplot(species_data, labels=iris['species'].unique())
axes[1, 1].set_title('Petal Width by Species')
axes[1, 1].set_xlabel('Species')
axes[1, 1].set_ylabel('Petal Width (cm)')

plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_5779/1564392205.py:27: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed '
  axes[1, 1].boxplot(species_data, labels=iris['species'].unique())
```




```
import numpy as np

arr1d = np.array([10, 20, 30, 40, 50])
print("1D Array:", arr1d)

arr2d = np.array([[1, 2, 3], [4, 5, 6]])
print("2D Array:\n", arr2d)

zeros = np.zeros((2, 3))
ones = np.ones((2, 3))
arange_arr = np.arange(1, 10, 2)
linspace_arr = np.linspace(0, 1, 5)

print("Shape of 2D Array:", arr2d.shape)
print("Data Type:", arr2d.dtype)
print("Dimensions:", arr2d.ndim)
print("Total Elements:", arr2d.size)

print("First Element of 1D Array:", arr1d[0])
print("Element at (1, 2) in 2D Array:", arr2d[1, 2])
print("Sliced 1D Array (Index 1 to 3):", arr1d[1:4])

arr = np.array([1, 2, 3, 4])
print("Addition (+10):", arr + 10)
print("Multiplication (*2):", arr * 2)

a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
```

```
print("Array Addition:", a + b)
print("Array Multiplication:", a * b)

matrix = np.array([[10, 20, 30], [40, 50, 60]])
print("Sum:", matrix.sum())
print("Mean:", matrix.mean())
print("Max:", matrix.max())
print("Min:", matrix.min())

reshaped = np.arange(1, 7).reshape(2, 3)
print("Reshaped Array (2x3):\n", reshaped)
```

```
1D Array: [10 20 30 40 50]
2D Array:
 [[1 2 3]
 [4 5 6]]
Shape of 2D Array: (2, 3)
Data Type: int64
Dimensions: 2
Total Elements: 6
First Element of 1D Array: 10
Element at (1, 2) in 2D Array: 6
Sliced 1D Array (Index 1 to 3): [20 30 40]
Addition (+10): [11 12 13 14]
Multiplication (*2): [2 4 6 8]
Array Addition: [11 22 33]
Array Multiplication: [10 40 90]
Sum: 210
Mean: 35.0
Max: 60
Min: 10
Reshaped Array (2x3):
 [[1 2 3]
 [4 5 6]]
```

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
    'Age': [25, 30, 35, 40, 28],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix'],
    'Salary': [70000, 80000, 120000, 110000, 65000],
    'Department': ['HR', 'IT', 'IT', 'Finance', 'HR']
}

df = pd.DataFrame(data)

print("First 5 Rows:\n", df.head())
print("\nDataset Shape:", df.shape)
print("\nDataset Info:\n")
df.info()
print("\nStatistical Summary:\n", df.describe())

print("\nSelected Columns (Name & Salary):\n", df[['Name', 'Salary']])
print("\nFiltered Data (Salary > 75000):\n", df[df['Salary'] > 75000])

df['Bonus'] = df['Salary'] * 0.10
print("\nDataFrame with New Column (Bonus):\n", df)

grouped = df.groupby('Department')['Salary'].mean()
print("\nAverage Salary by Department:\n", grouped)

sorted_df = df.sort_values(by='Salary', ascending=False)
print("\nSorted by Salary (Descending):\n", sorted_df)
```

```
First 5 Rows:
       Name  Age         City  Salary Department
0    Alice   25     New York   70000         HR
1      Bob   30  Los Angeles   80000         IT
2  Charlie   35      Chicago  120000         IT
3    David   40      Houston  110000    Finance
4      Eva   28      Phoenix   65000         HR

Dataset Shape: (5, 5)

Dataset Info:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Name        5 non-null      object
 1   Age         5 non-null      int64
```

```
 2   City        5 non-null      object
 3   Salary      5 non-null      int64
 4   Department  5 non-null      object
dtypes: int64(2), object(3)
memory usage: 332.0+ bytes

Statistical Summary:
             Age         Salary
count   5.00000       5.000000
mean   31.60000   89000.000000
std     5.94138   24596.747752
min    25.00000   65000.000000
25%    28.00000   70000.000000
50%    30.00000   80000.000000
75%    35.00000  110000.000000
max    40.00000  120000.000000

Selected Columns (Name & Salary):
       Name  Salary
0    Alice   70000
1      Bob   80000
2  Charlie  120000
3    David  110000
4      Eva   65000

Filtered Data (Salary > 75000):
       Name  Age         City  Salary Department
1      Bob   30  Los Angeles   80000         IT
2  Charlie   35      Chicago  120000         IT
3    David   40      Houston  110000    Finance

DataFrame with New Column (Bonus):
       Name  Age         City  Salary Department    Bonus
0    Alice   25     New York   70000         HR   7000.0
1      Bob   30  Los Angeles   80000         IT   8000.0
2  Charlie   35      Chicago  120000         IT  12000.0
3    David   40      Houston  110000    Finance  11000.0
4      Eva   28      Phoenix   65000         HR   6500.0
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = np.array([12, 15, 14, 10, 13, 12, 14, 100, 11, 15, 13, 12, 11, 14, 105, 12, 13, 10])
df = pd.DataFrame(data, columns=['Values'])

q1 = df['Values'].quantile(0.25)
q3 = df['Values'].quantile(0.75)
iqr = q3 - q1

lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr

iqr_outliers = df[(df['Values'] < lower_bound) | (df['Values'] > upper_bound)]['Values'].tolist()

mean = df['Values'].mean()
std = df['Values'].std()

z_scores = (df['Values'] - mean) / std
z_outliers = df[abs(z_scores) > 3]['Values'].tolist()

print(f"IQR Lower Bound: {lower_bound:.2f}, Upper Bound: {upper_bound:.2f}")
print("Outliers detected using IQR method:", iqr_outliers)

print(f"\nMean: {mean:.2f}, Standard Deviation: {std:.2f}")
print("Outliers detected using Z-score method (|Z| > 3):", z_outliers)

plt.figure(figsize=(12, 5))

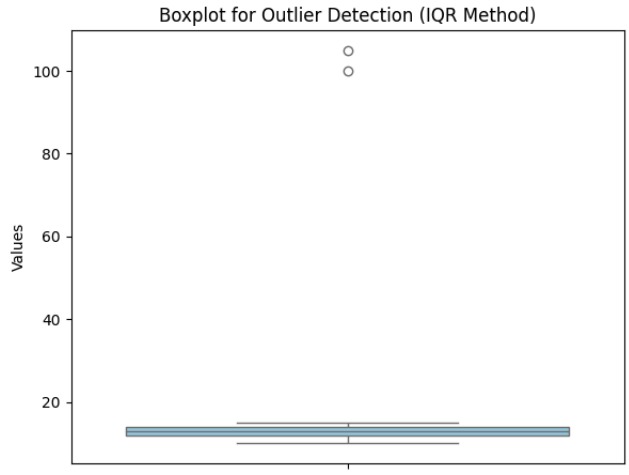
plt.subplot(1, 2, 1)
sns.boxplot(y=df['Values'], color='skyblue')
plt.title("Boxplot for Outlier Detection (IQR Method)")

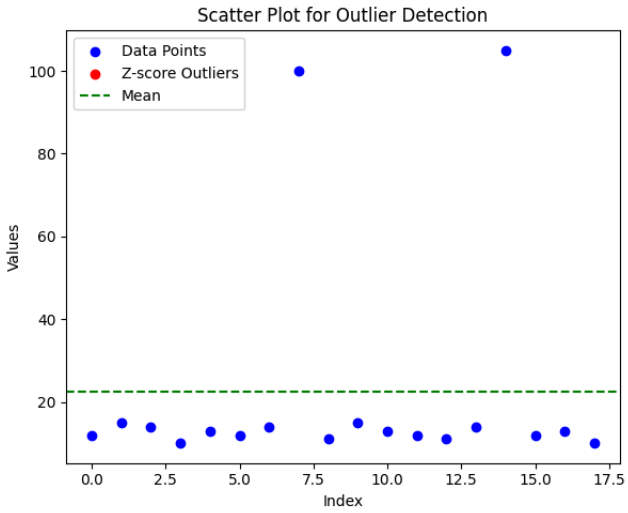
plt.subplot(1, 2, 2)
plt.scatter(df.index, df['Values'], color='blue', label='Data Points')
plt.scatter(df[abs(z_scores) > 3].index, z_outliers, color='red', label='Z-score Outliers')
plt.axhline(mean, color='green', linestyle='--', label='Mean')
plt.title("Scatter Plot for Outlier Detection")
plt.xlabel("Index")
plt.ylabel("Values")
plt.legend()

plt.tight_layout()
plt.show()
```

```
IQR Lower Bound: 9.00, Upper Bound: 17.00
Outliers detected using IQR method: [100, 105]

Mean: 22.56, Standard Deviation: 29.14
Outliers detected using Z-score method (|Z| > 3): []
```






```
import pandas as pd
import numpy as np

df = pd.read_csv('/content/gemini-code-1791880799261.txt')

df = df.drop_duplicates()

df['FoodPreference'] = df['FoodPreference'].replace({
    'veg': 'Veg',
    'Vegetarian': 'Veg',
    'non-Veg': 'Non-Veg'
})

df['Hotel'] = df['Hotel'].replace({'Ibys': 'Ibis'})

df['Rating(1-5)'] = df['Rating(1-5)'].apply(lambda x: np.nan if x < 1 or x > 5 else x)
df['NoOfPax'] = df['NoOfPax'].apply(lambda x: np.nan if x <= 0 else x)
df['EstimatedSalary'] = df['EstimatedSalary'].apply(lambda x: np.nan if x <= 0 else x)

df['Rating(1-5)'] = df['Rating(1-5)'].fillna(df['Rating(1-5)'].median())
df['NoOfPax'] = df['NoOfPax'].fillna(df['NoOfPax'].median())
df['EstimatedSalary'] = df['EstimatedSalary'].fillna(df['EstimatedSalary'].median())

print(df)
```

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  NoOfPax  \
0           1     20-25          4.0       Ibis            Veg  1300      2.0
1           2     30-35          5.0  LemonTree        Non-Veg  2000      3.0
2           3     25-30          4.0    RedFox             Veg  1322      2.0
3           4     20-25          4.0  LemonTree            Veg  1234      2.0
4           5       35+          3.0       Ibis            Veg   989      2.0
5           6       35+          3.0       Ibis        Non-Veg  1909      2.0
6           7       35+          4.0    RedFox             Veg  1000      2.0
7           8     20-25          4.0  LemonTree            Veg  2999      2.0
8           9     25-30          2.0       Ibis        Non-Veg  3456      3.0
9          10     30-35          5.0    RedFox        Non-Veg  6755      4.0

   EstimatedSalary
0          40000.0
1          59000.0
2          30000.0
3         120000.0
4          45000.0
5         122220.0
6          21122.0
7         345673.0
8          59000.0
9          87777.0
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
df=pd.read_csv('/content/gemini-code-1791908688839.txt')
```

```
X=df[['Age','Annual_Income']]
y=df['Purchased']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=42)
knn_unscaled=KNeighborsClassifier(n_neighbors=3)
knn_unscaled.fit(X_train,y_train)
acc_unscaled=accuracy_score(y_test,knn_unscaled.predict(X_test))
minmax=MinMaxScaler()
X_train_minmax=minmax.fit_transform(X_train)
X_test_minmax=minmax.transform(X_test)
knn_minmax=KNeighborsClassifier(n_neighbors=3)
knn_minmax.fit(X_train_minmax,y_train)
acc_minmax=accuracy_score(y_test,knn_minmax.predict(X_test_minmax))
scaler=StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
X_test_scaled=scaler.transform(X_test)
knn_standard=KNeighborsClassifier(n_neighbors=3)
knn_standard.fit(X_train_scaled,y_train)
acc_standard=accuracy_score(y_test,knn_standard.predict(X_test_scaled))
print(f"Accuracy without Scaling: {acc_unscaled*100:.2f}%")
print(f"Accuracy with MinMax Scaling: {acc_minmax*100:.2f}%")
print(f"Accuracy with Standardization: {acc_standard*100:.2f}%")
```

```
Accuracy without Scaling: 100.00%
Accuracy with MinMax Scaling: 100.00%
Accuracy with Standardization: 100.00%
```

```
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder
df=pd.read_csv('/content/gemini-code-1791908909431.txt')
print("Original Dataset:\n",df)
df=df.drop_duplicates()
num_imputer=SimpleImputer(strategy='median')
df['Salary']=num_imputer.fit_transform(df[['Salary']])
cat_imputer=SimpleImputer(strategy='most_frequent')
df['Gender']=cat_imputer.fit_transform(df[['Gender']]).ravel()
label_encoder=LabelEncoder()
df['Gender']=label_encoder.fit_transform(df['Gender'])
print("\nPreprocessed Dataset:\n",df)
```

```
Original Dataset:
    Age    Salary  Gender  Purchased
0   22   15000.0    Male          0
1   25   18000.0  Female          0
2   47       NaN  Female          1
3   52  145000.0    Male          1
4   23   22000.0    Male          0
5   40   85000.0     NaN          1
6   56  160000.0  Female          1
7   30   35000.0    Male          0
8   22   15000.0    Male          0
9   20   12000.0  Female          0

Preprocessed Dataset:
    Age    Salary  Gender  Purchased
0   22   15000.0       1          0
1   25   18000.0       0          0
2   47   28500.0       0          1
3   52  145000.0       1          1
4   23   22000.0       1          0
5   40   85000.0       0          1
6   56  160000.0       0          1
7   30   35000.0       1          0
9   20   12000.0       0          0
```

```
import pandas as pd
df=pd.read_csv('/content/gemini-code-1791909144130.txt')
num_cols=['Age','Salary']
cat_cols=['Department','Satisfaction_Level']
print("--- QUANTITATIVE ANALYSIS ---")
print("\nDescriptive Statistics:\n",df[num_cols].describe().T)
print("\nMean:\n",df[num_cols].mean())
print("\nMedian:\n",df[num_cols].median())
print("\nStandard Deviation:\n",df[num_cols].std())
print("\nCorrelation Matrix:\n",df[num_cols].corr())
print("\n--- QUALITATIVE ANALYSIS ---")
print("\nDepartment Frequencies:\n",df['Department'].value_counts())
print("\nSatisfaction Level Proportions:\n",df['Satisfaction_Level'].value_counts(normalize=True))
print("\nUnique Values per Categorical Column:\n",df[cat_cols].nunique())
print("\nMode of Categorical Features:\n",df[cat_cols].mode().iloc[0])
```

```
--- QUANTITATIVE ANALYSIS ---

Descriptive Statistics:
```

```
         count     mean           std      min       25%      50%       75%  \
Age      10.0     33.7      8.794569     23.0     28.25     31.0     38.75
Salary   10.0  58700.0  19601.870659  32000.0  48500.00  55000.0  71750.00

            max
Age        50.0
Salary  90000.0

Mean:
 Age          33.7
Salary    58700.0
dtype: float64

Median:
 Age          31.0
Salary    55000.0
dtype: float64

Standard Deviation:
 Age           8.794569
Salary    19601.870659
dtype: float64

Correlation Matrix:
             Age   Salary
Age     1.00000  0.98749
Salary  0.98749  1.00000

--- QUALITATIVE ANALYSIS ---

Department Frequencies:
 Department
IT         4
HR         3
Finance    3
Name: count, dtype: int64

Satisfaction Level Proportions:
 Satisfaction_Level
High      0.4
Medium    0.3
Low       0.3
Name: proportion, dtype: float64

Unique Values per Categorical Column:
 Department            3
Satisfaction_Level    3
dtype: int64

Mode of Categorical Features:
 Department              IT
Satisfaction_Level    High
Name: 0, dtype: object
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
df=pd.read_csv('/content/gemini-code-1791909344084.txt')
X=df[['Years_Experience']]
y=df['Salary']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
model=LinearRegression()
model.fit(X_train,y_train)
y_pred=model.predict(X_test)
print(f"Slope (m): {model.coef_[0]:.2f}")
print(f"Intercept (c): {model.intercept_:.2f}")
print(f"Mean Squared Error (MSE): {mean_squared_error(y_test,y_pred):.2f}")
print(f"R2 Score: {r2_score(y_test,y_pred):.4f}")
```

```
Slope (m): 9319.38
Intercept (c): 25782.03
Mean Squared Error (MSE): 52781578.82
R2 Score: 0.3654
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
df=pd.read_csv('/content/gemini-code-1791909468780.txt')
X=df[['Age','Estimated_Salary']]
y=df['Purchased']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=42)
scaler=StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
```

```
X_test_scaled=scaler.transform(X_test)
knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_scaled,y_train)
y_pred=knn.predict(X_test_scaled)
print(f"Accuracy: {accuracy_score(y_test,y_pred)*100:.2f}%")
```

```
Accuracy: 100.00%
Confusion Matrix:
 [[1 0]
 [0 2]]
```

```
import io
```